

DSAA2043: Exercise 5

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed array are accepted. However, when using Python, array indices must start from 0.

Problem 1. Let X be a sequence of n characters and Y be a sequence of m characters. A sequence Z is a *subsequence* of X if we can convert X to Z by repeatedly deleting zero or more symbols from X . A sequence Z is a *common subsequence* of X and Y if it is a subsequence of both. The *Longest Common Subsequence (LCS)* problem is to find a maximum length common subsequence between two sequences. Design an algorithm that finds the length of the LCS of X and Y . Your algorithm should run in $O(nm)$ time. For example, if $X = \langle A, B, C, B, D, A, B \rangle$, $Y = \langle B, D, C, A, B, A \rangle$, then one longest common subsequence is $\langle B, C, B, A \rangle$.

To formulate a dynamic programming solution, we focus on the prefixes $X[1 : i]$ and $Y[1 : j]$ and define a subproblem that captures the LCS of these prefixes. Let $\text{LCS}(i, j)$ denote the length of a longest common subsequence of these two prefixes. Understanding how $\text{LCS}(i, j)$ depends on smaller subproblems will allow us to design a dynamic programming algorithm that runs in $O(nm)$ time.

Answer the following questions:

- (a) Derive a recurrence for $\text{LCS}(i, j)$ in terms of smaller subproblems. Your recurrence should explicitly distinguish the cases $x_i = y_j$ and $x_i \neq y_j$.
- (b) Using your recurrence, fill in the $(n+1) \times (m+1)$ dynamic programming table for the following instance:

$$X = \langle C, A, D, B, A, C \rangle, \quad Y = \langle A, C, B, A, D \rangle.$$

	0	A	C	B	A	D
0						
C						
A						
D						
B						
A						
C						

- (c) From your completed table, state the length of a longest common subsequence of X and Y .
- (d) Trace back through the table to reconstruct one longest common subsequence. Write down the resulting sequence.

Answer 1(a): To derive the recurrence for $\text{LCS}(i, j)$, consider whether the last characters of the prefixes $X[1 : i]$ and $Y[1 : j]$ match.

- If $x_i = y_j$, then the last character contributes 1 to the LCS, and the remainder is the LCS of the two shorter prefixes:

$$\text{LCS}(i, j) = 1 + \text{LCS}(i - 1, j - 1).$$

- If $x_i \neq y_j$, then the longest common subsequence must end by discarding either x_i or y_j , so we take the larger of the two possibilities:

$$\text{LCS}(i, j) = \max(\text{LCS}(i - 1, j), \text{LCS}(i, j - 1)).$$

With base cases $\text{LCS}(0, j) = \text{LCS}(i, 0) = 0$, this recurrence completely defines the DP table.

Answer 1(b): Using the recurrence from part (a), we fill in the $(n + 1) \times (m + 1)$ DP table for the given sequences:

$$X = \langle C, A, D, B, A, C \rangle, \quad Y = \langle A, C, B, A, D \rangle.$$

Below is the completed dynamic programming table. Row i corresponds to $X[1 : i]$, and column j corresponds to $Y[1 : j]$.

	0	A	C	B	A	D
0	0	0	0	0	0	0
C	0	0	1	1	1	1
A	0	1	1	1	2	2
D	0	1	1	1	2	3
B	0	1	1	2	2	3
A	0	1	1	2	3	3
C	0	1	2	2	3	3

Answer 1(c): From the final row and column of the DP table, the length of a longest common subsequence of X and Y is **3**.

Answer 1(d): To reconstruct an LCS, we start from the bottom-right corner of the DP table and repeatedly apply the recurrence in reverse:

- If $x_i = y_j$, then the character is part of the LCS, and we move diagonally to $(i - 1, j - 1)$.
- If $x_i \neq y_j$, then we move to the neighbor $(i - 1, j)$ or $(i, j - 1)$ with the larger DP value.

Applying this procedure to the completed table, one possible reconstruction path yields the LCS

$$\langle C, A, D \rangle.$$

Depending on which equal-valued neighbor is chosen during ties, another valid LCS is

$$\langle A, B, A \rangle.$$

Any sequence of length 3 is a correct answer.

Problem 2. Given a set S of n items, with each item I_i has an associated weight w_i and value v_i . A knapsack with capacity W is also provided. The goal is to choose items with maximum total value but with weight at most W . If we are not allowed to take fractional amounts, this optimization problem is known as the *0/1-knapsack problem*. Formally, the goal is to compute

$$\max \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\} \text{ for } i = 1, \dots, n.$$

Design a dynamic programming algorithm that computes the optimal value of the 0/1-knapsack problem in $O(nW)$ time. For example, if the items are

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	8

and $W = 8$,

then the maximum total value of any feasible subset is 12, achieved for example by selecting items 2 and 4. **Answer the following questions:**

(a) Give the recurrence for $DP(i, c)$. Your recurrence should distinguish the cases where item i is excluded or included. State the base cases clearly.

(b) Using the following list of items and knapsack capacity

Item	Weight	Value
1	1	1
2	3	4
3	4	5
4	5	7

with capacity $W = 9$,

fill in the dynamic programming table $DP(i, c)$ for $i = 1, \dots, n$ and $c = 0, \dots, W$.

Rows correspond to items ($i = 1, \dots, n$), and columns correspond to capacities ($c = 0, \dots, W$). The entry $DP(i, c)$ stores the maximum value achievable using items $1 \dots i$ and capacity c .

Item i	Capacity c									
	0	1	2	3	4	5	6	7	8	9
1										
2										
3										
4										

(c) From your completed table, determine the maximum total value achievable under capacity $W = 8$. Also list the items included in one optimal solution.

Answer 2(a): We define $DP(i, c)$ to be the maximum total value achievable using only items $1, \dots, i$ with knapsack capacity c .

Base cases. If either no items are available or the capacity is zero, the optimal value is 0:

$$DP(0, c) = 0 \quad \text{for all } c = 0, \dots, W, \quad DP(i, 0) = 0 \quad \text{for all } i = 0, \dots, n.$$

Recurrence. For $i \geq 1$ and $c \geq 1$, we distinguish two cases depending on whether item i fits into capacity c :

- If $w_i > c$, then item i cannot be included. We must skip it:

$$DP(i, c) = DP(i - 1, c).$$

- If $w_i \leq c$, then we can either exclude or include item i . The optimal value is the better of these two choices:

$$DP(i, c) = \max(DP(i-1, c), DP(i-1, c-w_i) + v_i).$$

Together with the base cases, this recurrence allows us to fill the entire DP table in $O(nW)$ time.

Answer 2(b): For the instance

Item	Weight	Value
1	1	1
2	3	4
3	4	5
4	5	7

with capacity $W = 9$,

we use the recurrence from part (a) to fill the table $DP(i, c)$ for $i = 1, \dots, 4$ and $c = 0, \dots, 9$.

The completed DP table is:

Item i	Capacity c									
	0	1	2	3	4	5	6	7	8	9
1	0	1	1	1	1	1	1	1	1	1
2	0	1	1	4	5	5	5	5	5	5
3	0	1	1	4	5	6	6	9	10	10
4	0	1	1	4	5	7	8	9	11	12

Answer 2(c): From the final row and column of the DP table, we obtain $DP(4, 9) = 12$. Thus, under capacity $W = 9$, the maximum total value achievable is **12**.

To determine which items form an optimal solution, we trace back from the entry $DP(4, 9)$ in the DP table. At each step, if $DP(i, c) = DP(i-1, c)$, then item i is excluded; otherwise, item i is included and the traceback moves to $(i-1, c-w_i)$.

Applying this rule yields one optimal solution consisting of items 1, 2, and 4:

$$w_1 + w_2 + w_4 = 1 + 3 + 5 = 9 \leq W, \quad v_1 + v_2 + v_4 = 1 + 4 + 7 = 12.$$

Another optimal solution is to select items 3 and 4, which also achieves a total value of 12 with total weight $4 + 5 = 9$. Any subset with total value 12 earns full credit.

Problem 3. Let A be a sequence of n distinct integers. A sequence B of integers is a *subsequence* of A if it satisfies one of the following conditions:

- $A = B$, or
- we can convert A to B by repeatedly deleting integers.

The subsequence B is *ascending* if its integers are arranged in ascending order. Design an algorithm to find an ascending subsequence of A with the maximum length. Your algorithm should run in $O(n^2)$ time. For example, if $A = \langle 10, 5, 20, 17, 3, 30, 25, 40, 50, 60, 24, 55, 70, 58, 80, 44 \rangle$, then a longest ascending subsequence is $(10, 20, 30, 40, 50, 60, 70, 80)$.

Answer 3: We say that B is an *end-aligned ascending subsequence* of A if $A[n]$ is the last integer in B . In the example above, the subsequence $(5, 20, 30, 40, 44)$ is an end-aligned ascending subsequence

of A , while $(10, 20, 30, 40, 50, 60, 70, 80)$ is not. Given any $i \in [1, n]$, we use $\text{len}(i)$ to denote the maximum length of all end-aligned ascending subsequences of $A[1 : i]$. In our example, $\text{len}(16) = 5$, because $(5, 20, 30, 40, 44)$ is a longest end-aligned ascending subsequence of A , but $\text{len}(15) = 8$, because $(10, 20, 30, 40, 50, 60, 70, 80)$ is longest end-aligned ascending subsequence of $A[1 : 15]$.

Let B be an (arbitrary) longest end-aligned ascending subsequence of $A[1 : i]$, and define k to be the length of B . There are two possibilities:

- $k = 1$. This implies that $A[j] > A[i]$ for all $j < i$.
- $k > 1$. In this case, let j be the integer such that $B[k - 1] = A[j]$. Then $B[1 : k - 1]$ must be an end-aligned longest subsequence of $A[1 : j]$.

Given any $i \in [1, n]$, define $S(i) = \{j \mid j < i \text{ and } A[j] < A[i]\}$. The above discussion implies:

$$\text{len}(i) = 1 + \max_{j \in S(i)} \text{len}(j).$$

Using dynamic programming, we can compute $\text{len}(i)$ for all $i \in [1, n]$ in $O(n^2)$ time.

The maximum length of all ascending subsequences of A is

$$\max_{1 \leq i \leq n} \text{len}(i).$$

By the “piggyback technique”, we can produce a longest ascending subsequence of A in $O(n^2)$ extra time.

Problem 4. Given a set of n activities, where each activity i has a start time s_i and a finish time f_i . You can only do one activity at a time. The goal is to select the maximum number of activities whose time intervals do not conflict. This is known as the *activity selection problem*.

Consider the following instance:

Activity	Start (s_i)	Finish (f_i)
1	1	4
2	3	5
3	0	6
4	5	7
5	3	9
6	5	9
7	6	10
8	8	11
9	8	12
10	2	14

Answer the following questions:

(a) Write pseudocode for the greedy activity-selection algorithm. Assume that the activities have not been sorted in non-decreasing order of finish time.

(b) Using the example above, run your algorithm and list all activities selected by the greedy method.

(c) Analyze the running time and space usage of your algorithm. State the time complexity and space complexity using Big-O notation.

Answer 4(a): Because the activities are not initially sorted, the greedy algorithm begins by sorting all activities in non-decreasing order of finish time. Let the input be an array $A[1 \dots n]$, where $A[i].s$ and $A[i].f$ denote the start and finish times of activity i , respectively.

Algorithm 1 Greedy-Activity-Selection($A[1 \dots n]$)

```

1: Sort  $A$  in non-decreasing order of finish times.
2: Let  $S$  be an empty list of selected activities.
3:  $S.append(1)$  ▷ select the earliest-finishing activity
4:  $k \leftarrow 1$  ▷ index of the last selected activity
5: for  $i = 2$  to  $n$  do
6:   if  $A[i].s \geq A[k].f$  then
7:      $S.append(i)$ 
8:      $k \leftarrow i$ 
9:   end if
10: end for
11: return  $S$ 

```

The algorithm sorts the activities, selects the earliest-finishing activity, and then repeatedly selects the next activity whose start time is at least the finish time of the previously selected activity.

Answer 4(b): After sorting the activities by finish time, the order becomes:

(1 : [1, 4]), (2 : [3, 5]), (3 : [0, 6]), (4 : [5, 7]), (5 : [3, 9]), (6 : [5, 9]), (7 : [6, 10]), (8 : [8, 11]), (9 : [8, 12]), (10 : [2, 14])

Running the greedy algorithm yields the following decisions:

Activity	$[s_i, f_i]$	Condition	Decision
1	[1, 4]	First activity	Select
2	[3, 5]	$3 < 4$	Skip
3	[0, 6]	$0 < 4$	Skip
4	[5, 7]	$5 \geq 4$	Select
5	[3, 9]	$3 < 7$	Skip
6	[5, 9]	$5 < 7$	Skip
7	[6, 10]	$6 < 7$	Skip
8	[8, 11]	$8 \geq 7$	Select
9	[8, 12]	$8 < 11$	Skip
10	[2, 14]	$2 < 11$	Skip

Therefore, the greedy algorithm selects the following activities:

$$\boxed{\{1, 4, 8\}}.$$

Answer 4(c): Since the activities are not initially sorted, the algorithm begins by sorting them by finish time. Sorting requires $O(n \log n)$ time. After sorting, the algorithm performs one linear scan through the activities, making a constant-time decision at each step, which costs $O(n)$. Thus, the total running time is $O(n \log n)$.

Regarding space usage, the algorithm stores all n activities plus the output list of selected activities. Therefore, the total space complexity is $O(n)$.

Problem 5. Given n sticks, where stick i has a positive length ℓ_i . You can connect any two sticks of lengths a and b into a single new stick of length $a + b$, but the operation incurs a cost of $a + b$. After the merge, the new stick can be used in subsequent merges. The task is to combine all sticks into one single stick while minimizing the total cost of all merge operations.

Consider the following example:

Stick	Length (ℓ_i)
1	4
2	3
3	2
4	6

Answer the following questions:

(a) Write pseudocode for the greedy algorithm that uses a min-priority queue (min-heap) to repeatedly merge the two smallest sticks.

(b) Apply your algorithm step by step to the example above. At each iteration, identify the two sticks selected for merging, compute the cost of the merge, and record the length of the new stick inserted back into the min-heap. Finally, compute the total merge cost.

(c) Analyze the running time of the greedy algorithm assuming the initial list of stick lengths is not stored in a heap. Give your answer in Big-O notation and justify it.

Answer 5(a). Below is the pseudocode for the greedy algorithm using a min-priority queue (min-heap). The heap stores all current stick lengths.

Algorithm 2 Min-Cost-Connect-Sticks(L)

```

1: Build a min-heap  $H$  containing all stick lengths in  $L$ 
2: total_cost  $\leftarrow 0$ 
3: while  $H$  contains more than one stick do
4:    $a \leftarrow \text{Extract-Min}(H)$ 
5:    $b \leftarrow \text{Extract-Min}(H)$ 
6:    $c \leftarrow a + b$   $\triangleright$  merged stick length and cost
7:   total_cost  $\leftarrow$  total_cost +  $c$ 
8:   Insert( $H, c$ )
9: end while
10: return total_cost

```

Answer 5(b). We apply the algorithm to the example:

$\{4, 3, 2, 6\}$.

Step	Extracted sticks	Cost ($a + b$)	New heap contents
1	2, 3	5	$\{4, 5, 6\}$
2	4, 5	9	$\{6, 9\}$
3	6, 9	15	$\{15\}$

Total merge cost:

$$5 + 9 + 15 = 29.$$

Thus, the minimum total cost is 29.

Answer 5(c). The algorithm begins by building a min-heap from the n input stick lengths. This requires $O(n)$ time using the standard bottom-up heap construction procedure.

The main loop performs $n-1$ merge operations. Each iteration contains: 2 extract-min operations + 1 insert operation and each heap operation costs $O(\log n)$. Thus the total cost of the loop is $O(n \log n)$. Adding the initial heap construction, the overall running time is $O(n \log n)$.